

CSW306 — BACKEND DEVELOPMENT

Mẫu điền sẵn mang vào phòng thi

Quy trình đầy đủ SQL Server ⇌ Visual Studio ⇌ Swagger, kèm template code chứa chỗ điền và bảng quyết định dùng nhánh nào theo từng đề.

 **Quy ước:** chỗ nào viết `___TÊN_IN_HOA___` nghĩa là phải thay bằng giá trị thật lấy từ đề thi trước khi copy vào Visual Studio. Đọc kỹ Phần B (bảng quyết định) trước khi điền.

A. Quy trình đầy đủ: SSMS ⇌ Visual Studio ⇌ Swagger

Làm tuần tự từ trên xuống. Nhấn màu cho biết đang thao tác ở công cụ nào.

A1. Khởi tạo project (Visual Studio)

- [VS] 1. Mở Visual Studio → Create a new project → chọn "ASP.NET Core Web API" → Next.
- [VS] 2. Project name = `___DDMMYYYY_BE_HoTen___` (đúng convention đề yêu cầu) → chọn nơi lưu → Next.
- [VS] 3. Framework chọn .NET 8.0 (hoặc bản đề yêu cầu) → Authentication type: None → giữ nguyên "Use controllers" đã tick sẵn → Create.
- [VS] 4. Tools → NuGet Package Manager → Package Manager Console, chạy lần lượt từng dòng:

Package Manager Console

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer  
  
Install-Package Microsoft.EntityFrameworkCore.Tools  
  
Install-Package Microsoft.AspNetCore.Authentication.JwtBearer  
  
# Swashbuckle.AspNetCore (Swagger) thường có sẵn trong template, không cần cài thêm
```

- [VS] 5. Trong Solution Explorer, chuột phải vào tên project → Add → New Folder, tạo lần lượt: Models, Data, Repositories, Controllers, DTOs.

A2. Xác định SQL Server đang chạy (SSMS)

- [SSMS] 6. Mở SQL Server Management Studio → màn hình Connect to Server.
- [SSMS] 7. Server name: thử `___.` (dấu chấm) hoặc `___(localdb)\\MSSQLLocalDB___` hoặc `___.\SQLEXPRESS___` tùy máy đã cài bản nào → Connect.
→ Tên instance hiện ở góc trên Object Explorer sau khi connect thành công — copy đúng chuỗi này để dán vào connection string ở bước A4.
- [SSMS] 8. KHÔNG cần tạo Database thủ công bằng tay — Migration ở bước A6 sẽ tự tạo. Chỉ cần nhớ chọn 1 cái TÊN database sẽ dùng.

A3. Viết Model & ApplicationDbContext (Visual Studio)

- [VS] 9. Điền code Model theo Phần C1 vào Models/ — đối chiếu đúng bảng Field/Type/Constraints trong đề.
- [VS] 10. Điền ApplicationDbContext theo Phần C2 vào Data/.

A4. Cấu hình kết nối Database (Visual Studio)

- [VS] 11. Mở appsettings.json → điền connection string (Phần C2) với Server lấy từ bước A2, Database = tên bạn tự đặt.

[VS] 12. Mở Program.cs → điền đoạn AddDbContext + AddAuthentication JWT (Phần C2).

A5. Chạy Migration & kiểm tra ngược lại trong SSMS

[VS] 13. Package Manager Console, chạy:

```
Add-Migration InitialCreate
```

```
Update-Database
```

[SSMS] 14. Chuột phải vào "Databases" trong Object Explorer → Refresh.

[SSMS] 15. Mở rộng database vừa tạo → Tables → đếm đủ số bảng đúng số entity trong đề chưa.

⚠ Không thấy bảng nào? Quay lại VS, xem tab "Error List" hoặc output của Update-Database — thường do sai connection string ở A4. Sửa xong chạy lại Update-Database, KHÔNG cần Add-Migration lại.

A6. Repository & Controller (Visual Studio)

[VS] 16. Điền Repository theo Phần C3 (đọc Phần B để biết dùng bản cụ thể hay generic) vào Repositories/, đăng ký DI trong Program.cs.

[VS] 17. Điền Controller CRUD + CategoriesController theo Phần C4 vào Controllers/.

[VS] 18. Điền AuthController theo Phần C5 vào Controllers/.

[VS] 19. Điền OrdersController theo Phần C6 — đây là phần khó nhất, đọc kỹ Phần B trước khi điền.

A7. Tạo user test (SSMS) — trước khi chạy thử

[SSMS] 20. Trong Object Explorer, chuột phải đúng database vừa tạo → New Query.

[SSMS] 21. Dán câu lệnh INSERT (Phần C5) vào cửa sổ Query → bấm Execute (hoặc F5).

[SSMS] 22. Gõ SELECT * FROM Users; → Execute → kiểm tra 2 dòng user vừa tạo hiện ra đúng.

A8. Chạy thử & test trên Swagger

[VS] 23. Bấm nút Run màu xanh (hoặc F5) ở thanh công cụ trên cùng → trình duyệt tự mở Swagger UI (https://localhost:xxxx/swagger).

[SWAGGER] 24. Mở POST /api/auth/login → Try it out → dán JSON {"email":"admin@test.com","password":"..."} → Execute → copy giá trị "token" trong Response body.

[SWAGGER] 25. Bấm nút ổ khóa "Authorize" ở góc trên phải trang Swagger → gõ đúng cú pháp: Bearer ____dán_token_vào_đây____ → Authorize → Close.

[SWAGGER] 26. Test lần lượt từng endpoint GET/POST/PUT/DELETE, đối chiếu route trong đề. Endpoint public thử KHÔNG bấm Authorize trước để chắc chắn không cần token vẫn chạy.

[SSMS] 27. Sau khi POST/PUT/DELETE thử nghiệm, quay lại SSMS chạy SELECT để xác nhận dữ liệu đã lưu đúng vào bảng.

A9. Backup & nộp bài

[SSMS] 28. Chuột phải vào database → Tasks → Back Up... → Backup type: Full → Add đường dẫn lưu file .bak → OK.

[VS] 29. Đóng Visual Studio, zip toàn bộ folder project (có thể xóa bin/ và obj/ cho nhẹ) + file .bak vừa backup.

[VS] 30. Đặt tên file zip đúng convention đề yêu cầu → nộp bài.

B. Bảng quyết định — đề ghi gì thì dùng nhánh nào

Đọc kỹ đề trước khi điền code ở Phần C. Đề Q1/Q3/Lab7/Lab8 không hoàn toàn giống nhau ở những điểm sau — chọn đúng nhánh để không bị lệch yêu cầu.

Repository — cụ thể (specific) hay generic?

Dấu hiệu trong đề	Dùng cách nào
Đề liệt kê sẵn chữ ký: Task<List<X>> GetAll(); Task<X?> GetById(int id); ... (như Q1/Q3)	Dùng bản Repository CỤ THỂ (Phần C3 — bản A). Copy y chang tên method, KHÔNG thêm "Async" nếu đề không viết.
Đề ghi rõ: "Use the IRepository<T> interface" (như Lab 8)	Dùng bản GENERIC IRepository<T> (Phần C3 — bản B).

Role/quyền — tên chuỗi nào?

Dấu hiệu trong đề	Dùng cách nào
Bảng User trong đề ghi: Role NVARCHAR(20) NOT NULL: 'Admin' / 'User'	Dùng chuỗi "Admin" và "User" trong [Authorize(Roles="...")] — đúng chữ hoa/thường.
Bảng User ghi: Role ('Admin', 'Customer')	Dùng chuỗi "Admin" và "Customer" thay vì "User".

Response GET danh sách — nested hay flat?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi rõ chỉ 1 định dạng (VD chỉ nested {category: {...}})	Bám đúng định dạng đề yêu cầu, không tự đổi.
Đề ghi "(both formats accepted)" như Q1	Ưu tiên dùng FLAT (categoryId + categoryName) — code ngắn hơn, ít lỗi hơn nested.

GET orders/user/{userId} — Admin có được xem hộ không?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi: "userId in route must match userId in JWT token" — không nhắc ngoại lệ nào	KHÔNG cho Admin bypass ở route này. Admin xem hết đơn qua route riêng GET /api/orders (Admin-only).
Đề ghi rõ Admin được xem MỌI order kể cả qua route theo userId	Mới thêm nhánh kiểm tra Role == "Admin" để bypass — đọc đúng câu chữ, đừng tự suy diễn thêm quyền không được yêu cầu.

Password — có bắt buộc hash không?

Dấu hiệu trong đề	Dùng cách nào
Đề ghi: "Passwords can be stored as plain text for testing (no hashing)	Lưu plain text, KHÔNG cần cài BCrypt — tiết kiệm thời

Dấu hiệu trong đề	Dùng cách nào
required)"	gian, đúng yêu cầu.
Đề ghi: "Hash user passwords using BCrypt or ASP.NET Identity" là REQUIREMENT bắt buộc	Cài package BCrypt.Net-Next, hash password trước khi lưu và khi so sánh lúc login.

Đặt tên project khi tạo

Dấu hiệu trong đề	Dùng cách nào
Đề ghi format DateNow(DD-MM-YYYY)_BE_YourName	Đặt tên có DẤU GẠCH NGANG giữa ngày-tháng-năm đúng như mẫu, ví dụ 19-12-2025_BE_NguyenXuanCuong.
Đề ghi format DDMMYYYY_BE_YourName (không dấu gạch)	Đặt liền không dấu gạch, ví dụ 19122025_BE_NguyenXuanCuong.

C. Template code — chứa chỗ điền theo đề

C1. Model (`Models/\_\_\_TenEntity\_\_\_.cs`, `User.cs`, `Order.cs`, ...)

```
// Models/___TenEntity___.cs
public class ___TenEntity___
{
    public int Id { get; set; }

    [Required, MaxLength(___SoKyTuToiDa___)]
    public string ___TenField1___ { get; set; } // VD: Name

    [Required]
    [Column(ColumnName = "decimal(10, 2)")] // ĐÃ SỬA LỖI DECIMAL
    [Range(0.01, double.MaxValue, ErrorMessage = "___ThongBaoLoi___")]
    public decimal ___TenFieldGia___ { get; set; } // VD: Price

    // Thêm field khác theo đúng bảng trong đề tài, VD:
    // public ___KieuDuLieu___ ___TenField___ { get; set; }

    public int Quantity { get; set; } = 0; // Xóa nếu đề không yêu cầu

    [Required]
    public int ___TenBangChaId___ { get; set; } // VD: CategoryId

    [ForeignKey("___TenBangChaId___")]
    public ___TenLopCha___? ___TenBangCha___ { get; set; } // ĐÃ THÊM DẤU ? (NULLABLE)

    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

// Models/___TenLopCha___.cs
public class ___TenLopCha___ // VD: Category
{
    public int Id { get; set; }

    [Required, MaxLength(___SoKyTu___)]
    public string Name { get; set; }

    [MaxLength(255)]
    public string? Description { get; set; }

    public ICollection<___TenEntity___> ___TenEntity___s { get; set; } = new List<___TenEntity___>();
}

// Models/User.cs
public class User
{
    public int Id { get; set; }

    [Required, MaxLength(200)]
    public string FullName { get; set; }

    [Required, EmailAddress, MaxLength(255)]
    public string Email { get; set; }

    [Required]
```

```

    public string Password { get; set; }

    [Required]
    public string Role { get; set; } // "Admin"/"User" hoặc "Admin"/"Customer" - xem Phần B

    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

// Models/Order.cs
public class Order
{
    public int Id { get; set; }

    [Required]
    public int UserId { get; set; }

    [ForeignKey("UserId")]
    public User User { get; set; }

    public DateTime OrderDate { get; set; }

    [Column(TypeName = "decimal(10, 2)")]
    public decimal TotalAmount { get; set; }

    public string Status { get; set; } = "Pending";
    public DateTime CreatedAt { get; set; } = DateTime.Now;

    public ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
}

// Models/OrderItem.cs
public class OrderItem
{
    public int Id { get; set; }

    [Required]
    public int OrderId { get; set; }

    [ForeignKey("OrderId")]
    public Order Order { get; set; }

    [Required]
    public int __TenEntity__Id { get; set; } // VD: FruitId / PetId

    [ForeignKey("__TenEntity__Id")]
    public __TenEntity__ __TenEntity__ { get; set; }

    [Range(1, int.MaxValue)]
    public int Quantity { get; set; }

    [Column(TypeName = "decimal(10, 2)")]
    public decimal UnitPrice { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.Now;
}

```

C2. AppDbContext + appsettings.json + Program.cs

```
// Data/AppDbContext.cs
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<__TenEntity__> __TenEntity__s { get; set; }
    public DbSet<__TenLopCha__> __TenLopCha__s { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<__TenLopCha__>().HasIndex(c => c.Name).IsUnique();
        modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
        base.OnModelCreating(modelBuilder);
    }
}
```

```
// appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=__LayTuBuocA2__;Database=__TenDatabase__;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Jwt": {
    "Key": "___it_nhat_32_ky_tu_bat_ky_____",
    "Issuer": "ExamApi",
    "Audience": "ExamApiUsers",
    "ExpiryMinutes": 120
  }
}
```

```
// Program.cs
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;
using System.Text.Json.Serialization;

var builder = WebApplication.CreateBuilder(args);

// ĐÃ SỬA: Chống JSON Object Cycle khi Include quan hệ cha con
builder.Services.AddControllers().AddJsonOptions(options =>
{
    options.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;
});
builder.Services.AddEndpointsApiExplorer();

// ĐÃ SỬA: Thêm nút ổ khóa Authorize cho Swagger
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
```

```

        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Nhập: Bearer {token}"
    });
options.AddSecurityRequirement(new OpenApiSecurityRequirement
{
    {
        new OpenApiSecurityScheme
        {
            Reference = new OpenApiReference
            {
                Type = ReferenceType.SecurityScheme,
                Id = "Bearer"
            }
        },
        new string[] { }
    }
});
});

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// __DangKyRepositoryODay__ (xem Phan C3)
// builder.Services.AddScoped<I__TenEntity__Repository, __TenEntity__Repository>();

var jwtSettings = builder.Configuration.GetSection("Jwt");
var key = Encoding.UTF8.GetBytes(jwtSettings["Key"]);

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key)
    };
});
});
builder.Services.AddAuthorization();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

```

```

app.UseHttpsRedirection();
app.UseAuthentication(); // LUON truoc Authorization
app.UseAuthorization();
app.MapControllers();
app.Run();

```

C3-A. Repository — bản CỤ THỂ

```

// Repositories/I__TenEntity__Repository.cs
public interface I__TenEntity__Repository
{
    __SaoChepChuKyMethodTuDe__
    // VD: Task<List<Fruit>> GetAll();
    //     Task<Fruit?> GetById(int id);
    //     Task Add(Fruit fruit);
    //     Task Update(Fruit fruit);
    //     Task Delete(int id);
    //     Task<List<Fruit>> GetByCategoryId(int categoryId);
}

// Repositories/__TenEntity__Repository.cs
public class __TenEntity__Repository : I__TenEntity__Repository
{
    private readonly ApplicationDbContext _context;
    public __TenEntity__Repository(ApplicationDbContext context) => _context = context;

    public async Task<List<__TenEntity__>> GetAll() =>
        await _context.__TenEntity__.Include(x => x.__TenBangCha__).ToListAsync();

    public async Task<__TenEntity__?> GetById(int id) =>
        await _context.__TenEntity__.Include(x => x.__TenBangCha__)
            .FirstOrDefaultAsync(x => x.Id == id);

    public async Task Add(__TenEntity__ entity)
    {
        _context.__TenEntity__.Add(entity);
        await _context.SaveChangesAsync();
    }

    public async Task Update(__TenEntity__ entity)
    {
        _context.__TenEntity__.Update(entity);
        await _context.SaveChangesAsync();
    }

    public async Task Delete(int id)
    {
        var entity = await _context.__TenEntity__.FindAsync(id);
        if (entity != null)
        {
            _context.__TenEntity__.Remove(entity);
            await _context.SaveChangesAsync();
        }
    }

    public async Task<List<__TenEntity__>> GetByCategoryId(int categoryId) =>
        await _context.__TenEntity__.Where(x => x.__TenBangChaId__ == categoryId).ToListAsync();

```

```
}
```

C3-B. Repository — bản GENERIC (dùng khi đề yêu cầu IRepository<T>)

```
public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T?> GetByIdAsync(int id);
    Task<T> AddAsync(T entity);
    Task<T?> UpdateAsync(int id, T entity);
    Task<bool> DeleteAsync(int id);
}

public class __TenEntity__Repository : IRepository<__TenEntity__>
{
    private readonly AppDbContext _context;
    public __TenEntity__Repository(AppDbContext context) => _context = context;

    public async Task<IEnumerable<__TenEntity__>> GetAllAsync() =>
        await _context.__TenEntity__.ToListAsync();

    public async Task<__TenEntity__?> GetByIdAsync(int id) =>
        await _context.__TenEntity__.FindAsync(id);

    public async Task<__TenEntity__> AddAsync(__TenEntity__ entity)
    {
        _context.__TenEntity__.Add(entity);
        await _context.SaveChangesAsync();
        return entity;
    }

    public async Task<__TenEntity__?> UpdateAsync(int id, __TenEntity__ entity)
    {
        var existing = await _context.__TenEntity__.FindAsync(id);
        if (existing == null) return null;

        // ĐÃ SỬA TƯƠNG TỰ C4: Nên gán từng trường thay vì Update nguyên khối
        // __GanTungFieldODay__ (không dùng _context.Update() nguyên khối)

        await _context.SaveChangesAsync();
        return existing;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var existing = await _context.__TenEntity__.FindAsync(id);
        if (existing == null) return false;

        _context.__TenEntity__.Remove(existing);
        await _context.SaveChangesAsync();
        return true;
    }
}

// Đăng ký DI trong Program.cs:
// builder.Services.AddScoped<IRepository<__TenEntity__>, __TenEntity__Repository>();
```

C4. Controller CRUD chính + Categories

```
// Controllers/___TenEntity___sController.cs
[ApiController]
[Route("api/[controller]")]
public class ___TenEntity___sController : ControllerBase
{
    private readonly I___TenEntity___Repository _repo;
    private readonly AppDbContext _context;
    public ___TenEntity___sController(I___TenEntity___Repository repo, AppDbContext context)
    {
        _repo = repo; _context = context;
    }

    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetAll()
    {
        var list = await _repo.GetAll();
        var result = list.Select(x => new {
            x.Id, x.___TenField1___, x.___TenFieldGia___,
            ___TenBangChaId___ = x.___TenBangChaId___,
            ___TenBangCha___Name = x.___TenBangCha___?.Name
        });
        return Ok(result);
    }

    [HttpGet("{id}")]
    [AllowAnonymous]
    public async Task<IActionResult> GetById(int id)
    {
        var entity = await _repo.GetById(id);
        if (entity == null) return NotFound(new { message = $"___TenEntity___ {id} not found" });
        return Ok(entity);
    }

    [HttpPost]
    [Authorize(Roles = "___VaiTroAdmin___")] // xem Phan B: "Admin"
    public async Task<IActionResult> Create([FromBody] ___TenEntity___ entity)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        if (!await _context.___TenLopCha___s.AnyAsync(c => c.Id == entity.___TenBangChaId___))
            return BadRequest(new { message = "___TenBangChaId___ không tồn tại" });

        await _repo.Add(entity);
        return CreatedAtAction(nameof(GetById), new { id = entity.Id }, entity);
    }

    [HttpPut("{id}")]
    [Authorize(Roles = "___VaiTroAdmin___")]
    public async Task<IActionResult> Update(int id, [FromBody] ___TenEntity___ entity)
    {
        if (id != entity.Id) return BadRequest(new { message = "Id không khớp" });
        if (!ModelState.IsValid) return BadRequest(ModelState);

        var existing = await _repo.GetById(id);
        if (existing == null) return NotFound();

        // ĐÃ SỬA: Gỡ tracking object cũ để không bị EF core văng exception conflict Id
    }
}
```

```

        _context.Entry(existing).State = EntityState.Detached;

        if (existing.__TenBangCha__ != null)
        {
            _context.Entry(existing.__TenBangCha__).State = EntityState.Detached;
        }

        entity.__TenBangCha__ = null; // Tránh lỗi cascade update khi EF cố update bảng cha

        await _repo.Update(entity);
        return NoContent();
    }

    [HttpDelete("{id}")]
    [Authorize(Roles = "__VaiTroAdmin__")]
    public async Task<IActionResult> Delete(int id)
    {
        var existing = await _repo.GetById(id);
        if (existing == null) return NotFound();

        await _repo.Delete(id);
        return NoContent();
    }
}

// Controllers/__TenLopCha__sController.cs
[ApiController]
[Route("api/[controller]")]
public class __TenLopCha__sController : ControllerBase
{
    private readonly AppDbContext _context;
    public __TenLopCha__sController(AppDbContext context) => _context = context;

    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetAll() =>
        Ok(await _context.__TenLopCha__s.ToListAsync());
}

```

C5. AuthController + Tạo user test

```

// Controllers/AuthController.cs
[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly IConfiguration _config;
    public AuthController(AppDbContext context, IConfiguration config)
    {
        _context = context; _config = config;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginDto dto)
    {
        var user = await _context.Users.FirstOrDefaultAsync(u => u.Email == dto.Email);
        if (user == null || user.Password != dto.Password) // __doi_thanh_BCrypt.Verify_neu_de_bat_hash__

```

```

        return Unauthorized(new { message = "Sai email hoac password" });

var claims = new[]
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Email, user.Email),
    new Claim(ClaimTypes.Role, user.Role)
};

var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
var token = new JwtSecurityToken(
    issuer: _config["Jwt:Issuer"],
    audience: _config["Jwt:Audience"],
    claims: claims,
    expires: DateTime.Now.AddMinutes(double.Parse(_config["Jwt:ExpiryMinutes"])),
    signingCredentials: creds
);

return Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token), role = user.Role });
}
}

public class LoginDto { public string Email { get; set; } public string Password { get; set; } }

```

```

-- SQL – chạy trong SSMS (New Query trên đúng database)
INSERT INTO Users (FullName, Email, Password, Role, CreatedAt)
VALUES
('Admin Test', 'admin@test.com', 'admin123', '___VaiTroAdmin___', GETDATE()),
('User Test', 'user@test.com', 'user123', '___VaiTroUser___', GETDATE());
-- ___VaiTroAdmin___ / ___VaiTroUser___ = "Admin"/"User" hoặc "Admin"/"Customer", xem Phan B

SELECT * FROM Users;

```

C6. OrdersController — phần khó nhất

```

// Controllers/OrdersController.cs
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly AppDbContext _context;
    public OrdersController(AppDbContext context) => _context = context;

    [HttpGet]
    [Authorize(Roles = "___VaiTroAdmin___")]
    public async Task<IActionResult> GetAllOrders()
    {
        var orders = await _context.Orders.Include(o => o.OrderItems).ToListAsync();
        return Ok(orders);
    }

    [HttpPost]
    [Authorize]
    public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto dto)
    {
        var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
        if (userIdClaim == null) return Unauthorized();
    }
}

```

```

int userId = int.Parse(userIdClaim);

var order = new Order { UserId = userId, OrderDate = DateTime.Now, Status = "Pending" };
decimal total = 0;
foreach (var i in dto.Items)
{
    var entity = await _context.__TenEntity__s.FindAsync(i.__TenEntity__Id);
    if (entity == null)
        return BadRequest(new { message = $"__TenEntity__ {i.__TenEntity__Id} không tồn tại" });
    if (i.Quantity <= 0)
        return BadRequest(new { message = "Quantity phải > 0" });
    if (entity.Quantity < i.Quantity)
        return BadRequest(new { message = $"Không đủ hàng cho {entity.__TenField1__}" });

    var orderItem = new OrderItem
    {
        __TenEntity__Id = entity.Id,
        Quantity = i.Quantity,
        UnitPrice = entity.__TenFieldGia__ // giá HIEN TAI, không phải giá đã lưu sẵn
    };
    total += orderItem.Quantity * orderItem.UnitPrice;
    order.OrderItems.Add(orderItem);
    entity.Quantity -= i.Quantity;
}
order.TotalAmount = total;

_context.Orders.Add(order);
await _context.SaveChangesAsync();
return CreatedAtAction(nameof(GetOrderById), new { id = order.Id }, order);
}

[HttpGet("{id}")]
[Authorize]
public async Task<IActionResult> GetOrderById(int id)
{
    var order = await _context.Orders.Include(o => o.OrderItems).FirstOrDefaultAsync(o => o.Id == id);
    if (order == null) return NotFound();
    return Ok(order);
}

[HttpGet("user/{userId}")]
[Authorize]
public async Task<IActionResult> GetByUser(int userId)
{
    var tokenUserId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    if (tokenUserId != userId.ToString()) return Forbid();
    // __NEU_DE_CHO_PHEP_ADMIN_XEM_HO__: thêm điều kiện Role=="__VaiTroAdmin__" ở đây (xem Phần B)

    var orders = await _context.Orders.Include(o => o.OrderItems)
        .Where(o => o.UserId == userId).ToListAsync();
    return Ok(orders);
}
}

public class CreateOrderDto { public List<OrderItemDto> Items { get; set; } }
public class OrderItemDto { public int __TenEntity__Id { get; set; } public int Quantity { get; set; } }

```

3. Checklist "Bất Bại" 8 bước khi làm bài thi thực tế

1. **Kiểm tra NuGet:** Cài đủ 3 gói chính (Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools, Microsoft.AspNetCore.Authentication.JwtBearer).
2. **Server Name Connection String:** Nhớ dùng \\ nếu là instance có gạch chéo (ví dụ: (localdb)\\MSSQLLocalDB hoặc .\\SQLEXPRESS).
3. **Thứ tự Middleware trong Program.cs:** Luôn đảm bảo app.UseAuthentication(); nằm **TRƯỚC** app.UseAuthorization();.
4. **Đăng ký DI:** Viết xong Repository nào là vào ngay Program.cs thêm builder.Services.AddScoped<I...Repository, ...Repository>();.
5. **Cú pháp Authorize trên Swagger:** Khi copy token xong, bấm nút ở khóa góc trên phải, gõ đúng định dạng: Bearer <dấu cách><token>.
6. **Kiểm tra Role Case-Sensitive:** "Admin" khác "admin", "User" khác "Customer". Đọc đúng bằng User trong đề để set chuỗi role cho đồng nhất giữa DB, Token và [Authorize(Roles = "...")].
7. **Test Route Public vs Private:**
 - GET /api/pets, GET /api/categories: Không bấm Authorize vẫn phải ra dữ liệu.
 - POST /api/pets, DELETE /api/pets/{id}: Bắt buộc Authorize Admin mới cho phép.
8. **Backup Database:** Trước khi nộp bài nhớ thực hiện Task Back Up... database ra file .bak và gom vào zip nộp chung nếu đề yêu cầu.

1. Bảng "Từ Điển Thay Thế" (Find & Replace) cho đề này

Khi vào phòng thi gặp đề Fruits này, bạn chỉ cần thay các chỗ gạch dưới ____...____ trong file mẫu theo đúng bảng sau:

Chỗ chừa trống trong Cheat Sheet	Giá trị điền cho đề FRUITS này
____TenEntity____	Fruit
____TenEntity____s (số nhiều)	Fruits
____TenLopCha____	Category
____TenLopCha____s (số nhiều)	Categories
____TenBangChald____	CategoryId
____TenBangCha____	Category
____TenField1____	Name
____TenFieldGia____	Price
____VaiTroAdmin____	Admin
____VaiTroUser____	User

2. TOP 6 Lỗi hay gặp nhất khi copy code và cách xử lý ngay

- **Lỗi 1:** Gạch đổ toàn bộ file (Không nhận DbContext, JwtBearer, Authorize)
 - **Nguyên nhân:** Chưa cài NuGet Packages.
 - **Cách khắc phục:** Vừa tạo project xong, mở **Tools** → **NuGet Package Manager** → **Package Manager Console**, copy chạy ngay 3 dòng:

powershell

Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.AspNetCore.Authentication.JwtBearer

● Lỗi 2: Lỗi khi chạy Update-Database (Không kết nối được SQL)

- **Nguyên nhân:** Sai Connection String trong

appsettings.json.

- **Cách khắc phục:**

1. Mở SSMS, xem tên Server đang connect (thường là ., (localdb)\MSSQLLocalDB hoặc .\SQLEXPRESS).
2. Nếu tên có dấu gạch chéo \, trong appsettings.json **bắt buộc phải ghi 2 dấu **:

json

"DefaultConnection":

"Server=.\SQLEXPRESS;Database=FruitShopDB;Trusted_Connection=True;TrustServerCertificate=True"

● Lỗi 3: Gọi API FruitsController bị lỗi HTTP 500 (Unable to resolve service for type 'IFruitRepository')

- **Nguyên nhân:** Quên đăng ký Dependency Injection cho Repository trong

Program.cs.

- **Cách khắc phục:** Sau khi viết xong FruitRepository.cs, vào ngay Program.cs thêm dòng:

csharp

builder.Services.AddScoped<IFruitRepository, FruitRepository>();

● Lỗi 4: Đã đăng nhập lấy token nhưng gọi API vẫn bị 401 Unauthorized

- **Nguyên nhân 1:** Sai thứ tự middleware trong

Program.cs.

- **Quy tắc vàng:** app.UseAuthentication(); **PHẢI NẪM TRƯỚC** app.UseAuthorization();.
 - **Nguyên nhân 2:** Nhập sai cú pháp nút ổ khóa **Authorize** trên Swagger.
 - Phải gõ chữ: Bearer rồi mới dán token (có 1 dấu cách ở giữa): Bearer eyJhbGciOi...
-

● Lỗi 5: Lấy danh sách Orders bị crash JSON (A possible object cycle was detected)

- **Nguyên nhân:** Order chứa OrderItems, và OrderItem lại trỏ ngược về Order.
- **Cách khắc phục:** Trong

Program.cs, đoạn builder.Services.AddControllers() phải có cấu hình:

csharp

builder.Services.AddControllers().AddJsonOptions(options =>

{

options.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;

});

● **Lỗi 6: Update Fruit bị lỗi *The instance of entity type cannot be tracked***

- **Nguyên nhân:** Do EF Core track entity cũ từ repo rồi lại update object mới.
 - **Cách khắc phục:** Đã có sẵn trong template Phần C4 (chỉ cần giữ nguyên đoạn `_context.Entry(existing).State = EntityState.Detached;` và `entity.Category = null;`).
-

3. Timeline chiến thuật 120 phút trong phòng thi

title Kế hoạch 120 phút phòng thi CSW306

section Chuẩn bị & DB

Tạo Project & Cài 3 NuGet Packages : 0, 10

Viết Models & ApplicationDbContext (Q1 - 30đ) : 10, 25

Chạy Migration & Insert Test Users : 25, 35

section Repositories

Viết IFruitRepository & FruitRepository (Q2 - 30đ) : 35, 50

Đăng ký AddScoped trong Program.cs : 50, 55

section Controllers & JWT

AuthController & Login JWT : 55, 70

CategoriesController & FruitsController : 70, 85

OrdersController (Q3 - 40đ) : 85, 100

section Test & Nộp bài

Test Swagger toàn bộ endpoint : 100, 110

Backup DB (.bak) & Nén Zip nộp bài : 110, 120

4. Checklist nhanh trước khi bấm Run (F5)

1. Đã có 3 lệnh Install-Package thành công.
2. Đã chạy Add-Migration InitialCreate và Update-Database (vào SSMS thấy đủ 5 bảng).
3. Đã chạy script SQL INSERT INTO Users trong SSMS và thấy 2 dòng tài khoản.
4. Program.cs đã có `builder.Services.AddScoped<IFruitRepository, FruitRepository>();`.
5. `UseAuthentication()` đứng trước `UseAuthorization()`.
6. Đã thay toàn bộ các chữ `____...____` (không để sót placeholder nào).